preCICE [Google Summer of Code 2026]

Tejaansh Sara, thetejaansh@gmail.com

GitHub: brightyorcerf

Introduction thread on the preCICE Discourse forum (linked)

My initial experience with programming was freelancing websites for local clients with my friend in grade 9. Currently, I'm a 3rd year CS student at Mahindra University, Hyderabad. Most of my time is spent on building fun side-projects and competitive programming.

What motivates me to contribute to preCICE: Growing up, I spent hours watching veritasium and vsauce videos. Not a study requirement of course, but a genuine fascination with the way curiosity can unravel the physical world. I eventually picked CS over Engineering Physics but that foundational 'why' never truly left me: it simply changed its medium. preCICE felt like the missing bridge that intersected both my interests. Suddenly, the abstract rigor of C++ systems and the structural precision of a configuration pipeline had a tangible, scientific purpose.

Technical skills: #2501 - Implement RAII for xmlParserCtxt in ConfigParser (merged)

Reading ConfigParser::readXmlFile () to understand the SAX infrastructure, I caught an unchecked null return and exception-unsafe manual cleanup in the exact file the AST project will refactor. When fsimonis flagged a secondary corruption issue with xmlCleanupParser(), I re-read the libxml2 docs, removed it, and confirmed all tests pass before replying.

Communication skills: #2522 - inconsistencies in CouplingSchemeConfiguration (merged)

Demonstrates my ability to integrate into a professional development workflow and act on technical feedback. By strictly following the preCICE contribution guidelines, including the pre-commit hooks and markdown formatting, I ensured a low-friction merge into the 3.4.0 milestone.

Tools and time: Most of my time was invested in research: checking youtube workshop videos and preCICE discourse forum and poring over documentation to understand the context of the configuration pipeline.

Both PRs used the preCICE development environment (CMake, Boost.Test, pre-commit). For #2501 I spent around 3.5 hours exploring the libxml2 documentation, understanding the SAX callback lifecycle, writing the fix, and running tests. The xmlCleanupParser() issue took additional investigation. For #2522 I spent approximately 2 hours reading the coupling scheme configuration code and verifying the substeps attribute handling.

Other relevant PRs:

UN MUIOGO #302*: Atomic writes for resData.json* and *#326: Interrupted solver run detection*

The maintainer explicitly referenced my work in a discussion thread as an example of the kind of stability thinking the project needs

Project title: Clean multi-step configuration

What the project is based on:

Currently, it's like preCICE is trying to assemble a jigsaw puzzle while still grabbing pieces out of the box. More technically speaking, preCICE configuration pipeline couples XML parsing and object construction in a single recursive pass through ConfigParser::connectTags(). This design (documented in #982 by fsimonis) means that domain objects are built while parsing is still in progress. I propose breaking this monolith into a clean, four-stage pipeline:

- Raw Parsing: libxml2 generates a raw XML AST.
- Typed Transformation: We convert that raw data into a structured Configuration AST with dedicated types like config::Mesh and config::Participant.
- Validation Pass: A dedicated checker ensures all cross-entity relationships are valid *before* a single object is built.
- Final Construction: The validated AST is used to instantiate the actual preCICE objects.

Benefit for the community:

The current single-pass architecture makes it impossible to validate a configuration file completely before construction begins, this means that users encounter errors mid-construction (rather than receiving a comprehensive diagnostic). My Configuration AST decouples these concerns, enabling 3 concrete improvements that users will feel immediately:

- all configuration errors reported at once.
- order-independent processing (a <mesh> tag referencing a <data> defined later in the file will work correctly).
- the foundation for GCC-style error messages with file line numbers, a long-requested feature tracked in issue #751.

As for developers, the separation makes each stage independently testable and debuggable.

Why personally interested:

My interest in this project is a direct result of "living" inside the ConfigParser code. As a third-year CSE student, I am intentionally seeking out this refactor because I believe true engineering growth happens at the heart of a library. Navigating that unknown is where the real work (and the real fun) starts. I want to push my limits by replacing this legacy bottleneck with a modern, diagnostic-rich pipeline.

PROJECT PLAN & TIMELINE

Related: This project surgically replaces preCICE's configuration heart. It is grounded in the #982 blueprint (fsimonis's typed-struct design) and #751 (GCC-style errors). My experience with contributions already will help me hit the ground running. With 24/7 availability during holidays, I will maintain a high-velocity strategy.

Phase 1: Foundation & Architectural Audit (May 24 — June 21)

M1: Characterization (Weeks 1-2): Write characterization tests against connectTags() (ConfigParser.cpp:225–275) to capture all the regression baselines and coupling points.

M2: Metadata-Rich AST (Weeks 3-4): Audit 13 listener classes to map 50+ attributes to typed-struct members, ensuring 1:1 parity. Every struct inherits from a base ConfigNode storing SourceLocation. I will implement a SAX-to-AST state machine to maintain parent-child hierarchy during the streaming parse. Sequence-preserving containers will be used for CouplingScheme to address order-dependency (#982). Deliverable**:** ConfigurationAST.hpp (~150 LOC).

Key files ConfigParser.cpp:225–275 (connectTags), ConfigParser.cpp:277 (xmlSAX2GetLineNumber()), XMLAttribute.hpp:160 (readValue), XMLTag.cpp:255–271 (resetAttributes hack).

Phase 2: Transformation & Diagnostic Parity (June 22 - July 12 | Mid-term evaluation)

M3: XML → AST Pipeline: Implementing buildConfiguration (CTag root, XMLTag schema). This will be a pure function with no side effects. Diagnostic Parity: The pass traverses the CTag tree, populating an ErrorAccumulator by reading SourceLocation metadata already persisted into the nodes during the initial SAX parse. Success is defined by reporting multiple configuration errors in a single pass with exact file context, ending the "fail-on-first-error" limitation.

The ErrorAccumulator manages a std::vector<Diagnostic> (Location, Severity, Message), enabling non-stop collection: Phase 2 continues sibling traversal after errors to report all issues at once. A "hard gate" halts the pipeline before Phase 3 (Construction) if fatal errors exist, ensuring a side-effect-free validation stage.

Phase 3: Safety Net & Integration (July 13 - August 24 | Final)

M4: Shadow Execution: Before deleting legacy code, I will run both pipelines in parallel. I'll use PRECICE_ASSERT to verify that the config hashes of the new AST perfectly match the legacy output. This proves the new pipeline is a drop-in replacement.

M5: Construction Pass: Replace connectTags() with: buildConfiguration() → CheckerPass → ConstructionPass. Since the new pass only visits nodes requiring instantiation, we eliminate the need for these architectural dummy classes entirely. To keep the 13 listeners untouched, the ConstructionPass acts as an Adapter/Driver, iterating the AST to trigger existing listener methods. This formally removes the NoPListener hack while preserving physics logic.

M6: Order Independence: With the full AST available, I will implement order-independent processing (e.g., <mesh> referencing <data> defined later), fully closing #982. I will structure checkers for cross-entity validation (unique data names, participant existence).

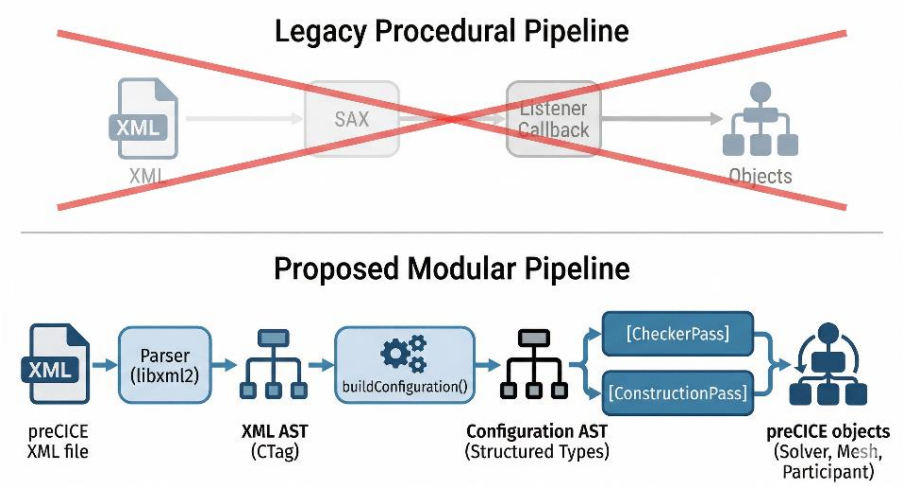
Phase 4: Extended Goals (August 24 - November)

Stretch Goal: Full compiler-style output with caret underlining (^~~~) using _fileContent stored as a member.

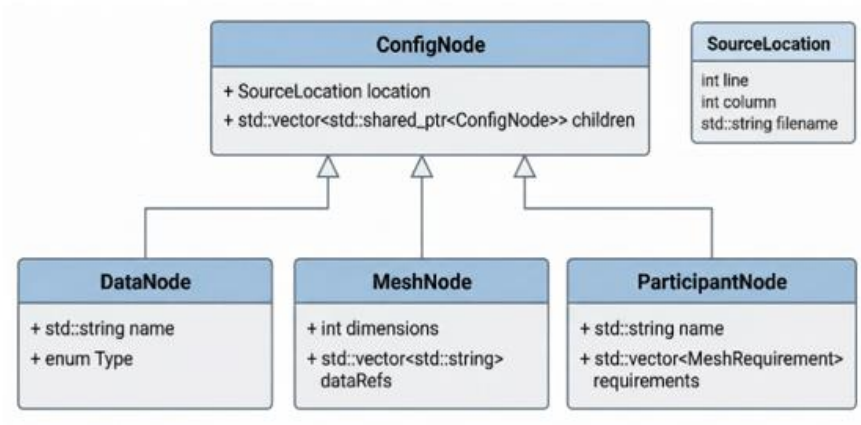
Expected Results: Zero regressions across 553 tests; a modular, testable AST pipeline; and GCC-style error messaging that eliminates the trial-and-error configuration loop.

ConfigParser.cpp (lines 264-274)

```
264
265     pDefSubTag->_configuredNamespaces[pDefSubTag->_namespace] = true;
266     pDefSubTag->readAttributes(subtag->m_aAttributes);
267     pDefSubTag->_listener.xmlTagCallback(context, *pDefSubTag);
268     pDefSubTag->_configured = true;
269
270     connectTags(context, pDefSubTag->_subtags, subtag->m_aSubTags);
271
272     pDefSubTag->areAllSubtagsConfigured();
273     pDefSubTag->_listener.xmlEndTagCallback(context, *pDefSubTag);
```



Transitioning from a "fail-on-first-error" single-pass (Top) to a decoupled, 4-stage AST pipeline (Bottom) for global validation.



ConfigNode Hierarchy: By deferring construction, nodes inherit SourceLocation metadata, providing the "nervous system" for GCC-style diagnostics and multi-pass validation.